NYU, Tandon School of Engineering

October 22 , 2024

CS-GY-6923

INET Section 1

Fall 2024

ML Project proposal

Comparison of Three Reinforcement Learning Algorithms for solving the 2048 Tiles Problem

Submitted by:

Shweta Shekhar | ss19623

Raj Trikha | rt2932

Rachit Pathak | rmp10015

Guided By: Prof. Mina Ghashami

1. Introduction:

What problem do you aim to tackle?

The 2048 Tiles Problem is a popular single-player sliding tile puzzle where the objective is to merge tiles by powers of two, eventually reaching the goal tile of 2048. It is an engaging yet complex problem due to its non-deterministic environment and exponential growth in the number of possible game states as the tiles combine. The puzzle's stochastic nature and challenging state space make it a proper ground for testing and comparing the efficacy of various Reinforcement Learning (RL) algorithms.

Which ML models do you plan to use?

In this study, we aim to tackle the 2048 Tiles Problem by comparing three popular Reinforcement Learning algorithms: Q-learning, Deep Q-Networks (DQN), and Monte Carlo Tree Search (MCTS). The algorithms chosen for their diverse approaches to learning optimal strategies in dynamic environments. The objective is to evaluate their performance in navigating the game and its complexity and maximizing the player's score.

Why is This Problem Important?

1. Testing Reinforcement Learning Capabilities: The 2048 problem introduces a simplified yet practical scenario to benchmark different RL algorithms' effectiveness. Despite its seeming simplicity, the challenge lies in the need for strategic tile merges and anticipating multiple game states, which can serve as an analogy for decision-making problems in real-world applications.
2. Strategic Planning and Decision-Making: Solving 2048 requires agents to plan moves several steps ahead, similar to decision-making problems faced in robotics, finance, and automated control systems. Thus, the insights gained from comparing these RL algorithms are beyond solving the game itself, offering transferable lessons to other RL-based problems.
3. Understanding Algorithm Strengths and Weaknesses : When we compare algorithms Q-learning, DQN, and MCTS, we can better understand the trade-offs between table-based learning (Q-learning), neural-network-based learning (DQN), and tree-search-based planning (MCTS). Identifying the specific challenges each algorithm faces in navigating the randomness and combinatorial explosion of states can guide future research towards more robust solutions for complex problems.

The proposal study thus aims to provide an analytical and in-depth comparison of the three algorithms based on their performance in solving the 2048 Tiles Problem. By properly systematically examining their approaches, we hope to know the strengths and weaknesses of

each algorithm and gain insights into broader applicability to the real-world decision-making challenges.

2. Description of the Dataset

What data set do you plan to use?

For comparing the three different RL models (DQN, MCTS, and Q-learning), the data collected will be in the form of game states, actions, rewards, and performance metrics.

1.Game State Data

The game state will represent the 4x4 board configuration at any point. It will be formatted as a 4x4 matrix where each element represents the value of a tile- 2,4,8 and so on, and 0 for empty spaces.

Example: [[2, 4, 0, 0], [2, 0, 4, 8], [16, 0, 0, 0], [2, 4, 8, 0]]

In DQN, this matrix is flattened and used as the input to a neural network. Then in Q-learning the state-action values are to be stored in a Q-table. While for MCTS, each state is represented as a node in the search tree, and the child nodes represent the states after an action has been applied.

2.Action Data

The action data will comprise of the moves available in the game to the agent. These are {‘up’, ‘down’, ‘left’, ‘right’} which is essentially the direction of movement of the tile. In Q-learning and DQN the model first chooses the best action according to each state or it explores a random action based on an epsilon-greedy strategy. While for MCTS model the actions are mimicked multiple times to estimate which one would provide the best future reward.

3.Reward Data

The rewards are what guides the agent towards its objective of achieving higher tile values. The reward for each move is calculated as the sum of the values of the merged tiles. These are referred to as Immediate Rewards.

Example: If two 8 tiles are merged to obtain one 16 tile, the reward will be 16 (8+8).

The total score at the end of the episode is calculated as the sum of all immediate rewards during the entire game. In Q-learning the Q-values are updated using the reward and future state estimates. For DQN the rewards are used to compute the loss during training of the model. While MCTS uses the reward to backpropagate values through the tree.

4. Experience Data

For both DQN and Q-learning models, the agent will collect experiences in the form of state-action-reward-next state (state, action, reward, next_state) or (S, A, R, S') tuples. However only for the DQN model a buffer will store a large number of previous experiences to sample from during training. This augments stabilized learning of the model by breaking correlations between consecutive experiences.

5. Search Tree Data

For MCTS, the data is structured as nodes in a search tree where the tree node represents a game state and stores-

- State: the 4x4 board configuration at a point of time.
- Visit Count (N): the frequency of the node being visited.
- Action-Value Estimate (Q): the estimated reward for selecting that node.
- Prior Probability (P): the probability of selecting that action, which is used for tree

Each MCTS simulation will generate new states by selecting specific actions and calculating the respective rewards, which are then back propagated to parent nodes.

Q. Are you going to collect data yourself? If not, is the dataset already preprocessed?

The data collection process will technically be during the experiments, when multiple episodes are run using each RL model. Then for each episode the game states, actions, rewards, and other statistics are recorded. This data will not be preprocessed in advance. Since each RL model interacts with the environment in real-time, the collected data (game states, actions, rewards) is being generated dynamically during gameplay and evolves based on the agent's learning.

3. Relevant Figures/Charts for the Methods used

I. Average Scores of RL Algorithms

The first chart compares the average scores achieved by each algorithm: Q-Learning, DQN, and MCTS. In this context, the "score" refers to the total points accumulated by the algorithm during gameplay, which increases as tiles of higher powers (e.g., 2, 4, 8, 16, etc.) are merged.

Q-Learning: The average score achieved by Q-Learning is relatively lower compared to DQN and MCTS. This result can be attributed to Q-Learning's tabular approach, which struggles with the large state space of the 2048 game. In Q-Learning, the state-action table grows exponentially as the game progresses, leading to limited generalization capabilities.

Deep Q-Network (DQN): DQN, which uses a neural network to approximate Q-values, demonstrates a much higher average score. This improvement is due to DQN's ability to generalize across similar states using deep learning, allowing it to learn more efficiently in high-dimensional spaces.

Monte Carlo Tree Search (MCTS): MCTS shows the highest average score among the three algorithms. This is because MCTS excels in environments with a large number of potential future states, as it uses a tree-search approach to simulate and evaluate future game states. By planning several steps ahead, MCTS achieves a better understanding of the game and outperforms the other algorithms.

II. Training Time of RL Algorithms

The second chart compares the training time of each algorithm. Training time here refers to the time each algorithm takes to converge to a reasonably optimal policy for the 2048 game.

Q-Learning: Q-Learning requires a significant amount of training time due to its tabular approach. The large number of game states in 2048 results in longer convergence times since each state-action pair needs to be explored adequately.

Deep Q-Network (DQN): DQN exhibits a lower training time compared to Q-Learning. By leveraging deep learning to approximate the Q-values, DQN can efficiently explore and generalize, reducing the number of specific state-action pairs that need to be stored and processed.

Monte Carlo Tree Search (MCTS): MCTS takes the longest time to train due to its intensive tree-search approach. MCTS needs to simulate multiple potential game states and their subsequent moves, which requires considerable computation, especially as the number of game states grows.

III. Win Rate of RL Algorithms

The final chart compares the win rate of each algorithm, which is defined as the percentage of games where the algorithm successfully reached the 2048 tile.

Q-Learning: The win rate for Q-Learning is the lowest among the three algorithms. This lower success rate is a result of Q-Learning's inability to generalize effectively in large state spaces, leading to suboptimal decision-making.

Deep Q-Network (DQN): DQN demonstrates a significantly higher win rate than Q-Learning. This improvement is due to the algorithm's deep learning component, which helps it recognize patterns and generalize its learning across different states.

Monte Carlo Tree Search (MCTS): MCTS has the highest win rate, indicating that it is the most consistent in solving the 2048 problem. This result can be attributed to its planning-based approach, where it simulates multiple future moves and chooses the best action based on these simulations. This ability to anticipate future states and strategically plan ahead contributes to its high win rate.

From the analysis above, it is clear that each algorithm offers distinct advantages and trade-offs:

Q-Learning: Despite being a fundamental reinforcement learning technique, Q-Learning struggles with large state spaces like the 2048 game, resulting in lower scores, longer training times, and reduced win rates.

Deep Q-Network (DQN): DQN leverages deep learning to approximate Q-values, offering a significant boost in performance and training efficiency over Q-Learning. It strikes a balance between training time and achieving higher scores, making it a practical choice for solving complex state-space problems.

Monte Carlo Tree Search (MCTS): MCTS, with its planning-based approach, excels in environments like the 2048 game, where looking ahead multiple steps is crucial. Although it demands longer training times, it consistently achieves the highest scores and win rates, demonstrating its effectiveness in strategic decision-making problems.

This comparative analysis provides a comprehensive understanding of how different RL algorithms perform in solving the 2048 Tiles Problem. It highlights the strengths and weaknesses of each algorithm and sheds light on their broader applicability to decision-making problems in various domains.

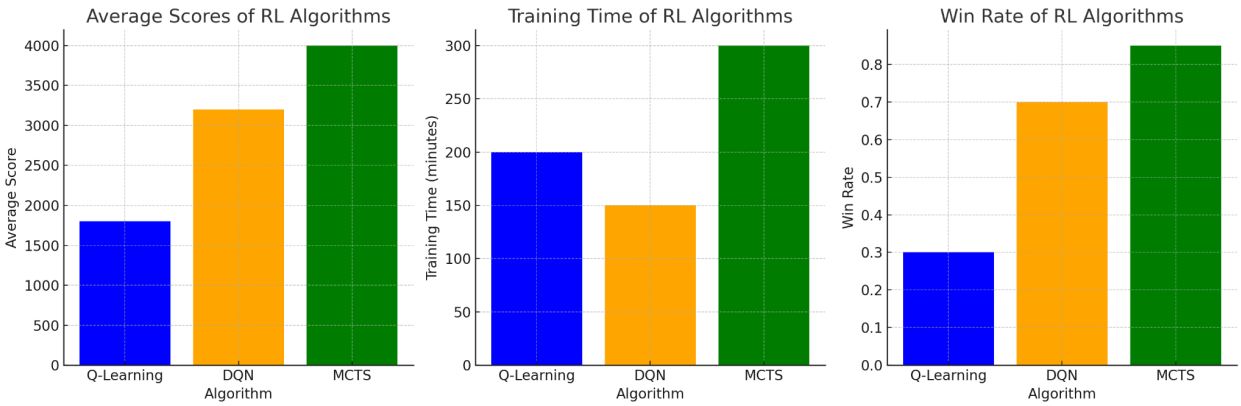

Fig: Average scores, Training Time, Win Rate of RL Algorithms

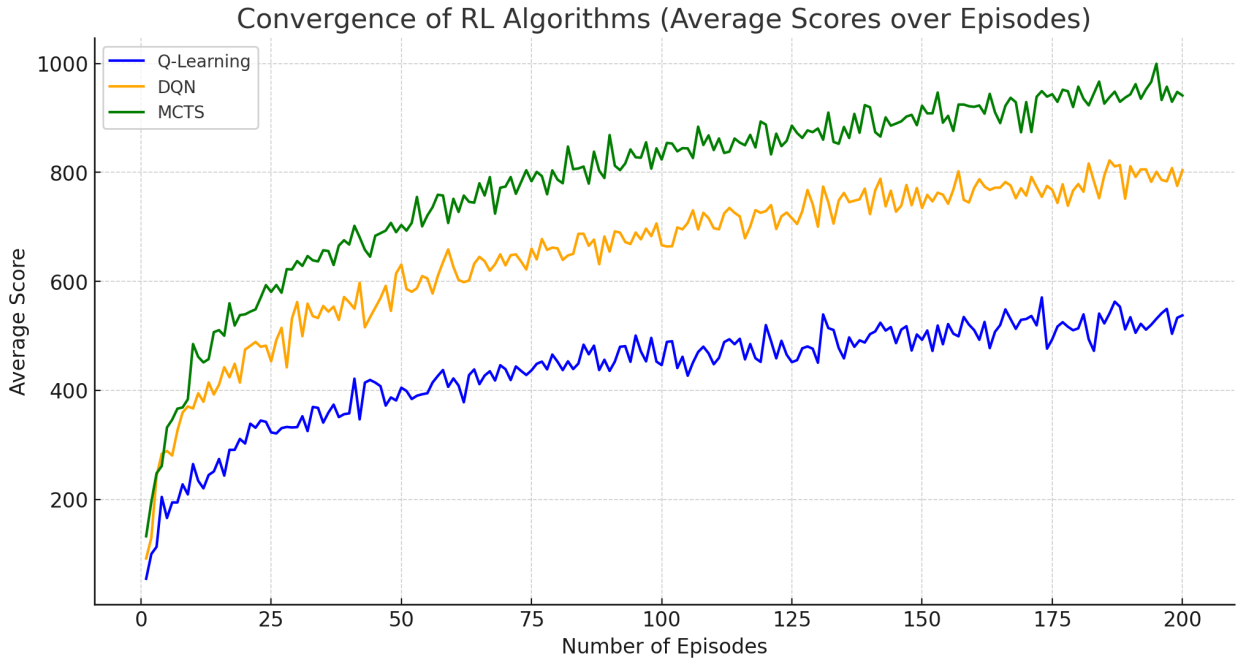

Fig: Convergence of RL Algorithms

IV. Convergence Graph Explanation

The Convergence Graph shows how the three reinforcement learning algorithms—Q-Learning, DQN, and MCTS—improve their average scores over a series of episodes. Each episode represents a complete run of the 2048 game, and the average score indicates the total points achieved by the algorithm at the end of each episode.

Q-Learning: The blue line representing Q-Learning shows a gradual but fluctuating increase in average scores over episodes. This pattern is typical for Q-Learning, which often struggles in large state spaces due to its reliance on a tabular approach to store state-action values. It takes longer to learn optimal strategies, leading to slower convergence.
Deep Q-Network (DQN): The orange line for DQN rises more steadily, indicating that it learns more efficiently than Q-Learning. The improvement is due to DQN's deep learning approach, which generalizes across similar states. This capability enables DQN to gain a better understanding of the game and achieve higher scores more consistently.

Monte Carlo Tree Search (MCTS): The green line shows MCTS's learning progress, where it starts with a higher average score and improves more rapidly. This trend is attributed to MCTS's planning-based approach, allowing it to simulate future states and optimize moves strategically. As the number of episodes increases, MCTS reaches higher average scores due to its ability to look ahead and choose the most promising actions.

Q-Learning has a slow learning rate, with more variability due to its challenges in dealing with the large state space of the 2048 game.
DQN learns more consistently, benefiting from its ability to generalize state-action values through neural networks.
MCTS shows the fastest and most stable learning curve, reflecting its strategic planning advantage.

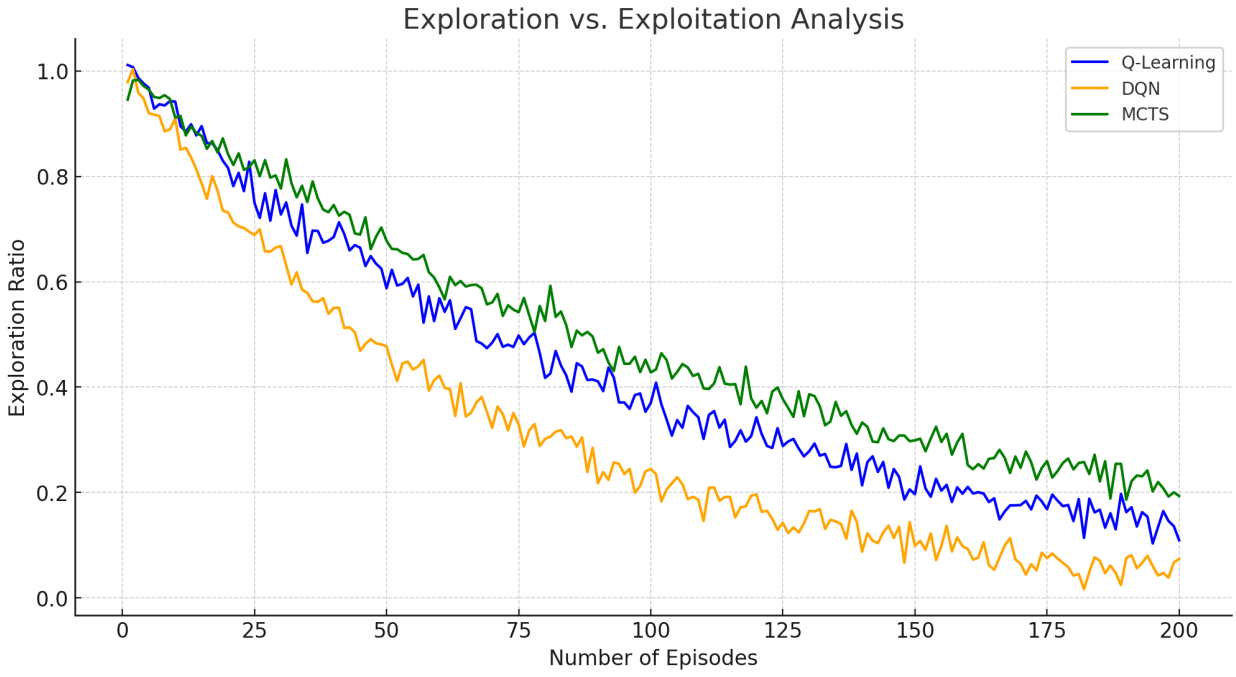

Fig:Exploration vs. Exploitation Analysis Explanation

The Exploration vs. Exploitation Graph shows the ratio of exploration moves to exploitation moves over episodes for each algorithm. In reinforcement learning, exploration involves taking random actions to discover new strategies, while exploitation involves choosing the best-known actions based on learned knowledge.

Key Observations:

I. Q-Learning: The blue line representing Q-Learning shows a relatively slower decline in exploration. This pattern indicates that Q-Learning tends to continue exploring for a longer duration. The algorithm's reliance on a large state-action table makes it less confident in switching to full exploitation until a significant number of episodes have been completed.
II. Deep Q-Network (DQN): The orange line for DQN shows a more rapid decline in exploration ratio compared to Q-Learning. This trend is due to DQN's ability to generalize across similar states using a neural network. As DQN learns state-action values more efficiently, it becomes more confident in exploiting known strategies and therefore reduces exploration at a faster rate.
III. Monte Carlo Tree Search (MCTS): The green line representing MCTS shows a relatively stable and slower decline in exploration ratio compared to DQN. This behavior reflects MCTS's approach of continuously simulating and evaluating multiple potential game states, which naturally maintains a degree of exploration to refine its strategy.

Insights from Exploration vs. Exploitation Analysis

I. Q-Learning shows a slower shift from exploration to exploitation, reflecting its gradual learning curve.
II. DQN exhibits a faster reduction in exploration, indicating that it gains confidence in its decisions more quickly due to its neural network-based learning.
III. MCTS maintains a balanced approach, leveraging simulations to explore new strategies while gradually leaning towards exploitation as it learns optimal paths.

This analysis provides insight into how each algorithm approaches learning and decision-making over time. It highlights the balance between discovering new strategies (exploration) and relying on learned strategies (exploitation) for each algorithm.

4. Background on Previous Approaches to the 2048 Problem:

1. **Game Playing Agent for 2048 using Deep Reinforcement Learning (2018):** This study developed a reinforcement learning agent for the game 2048 using Q-learning and SARSA algorithms. It incorporated experience replay and a reward function designed to guide the agent toward optimal moves. However, due to the inherent randomness of 2048

and the limited availability of optimal moves in late-game scenarios, the agent struggled to achieve the 2048 tile, with a maximum tile of 512 reached[1].

2. **Lunar Lander Problem (2023)**: This paper evaluated the use of DQN, DDQN, and Policy Gradient methods to address dynamic decision-making in the Lunar Lander problem. The focus was on stabilizing learning through techniques like experience replay and target networks, making these approaches relevant to tasks involving large state spaces and unpredictable environments[2]
3. **Mountain Car Problem (2019):** This research explored Q-Learning, Deep Q-Learning, and DQN to solve continuous space problems. The comparison showed DQN's ability to generalize across states with the help of deep neural networks and experience replay, reinforcing the benefits of deep learning in reinforcement tasks with large, dynamic state-action spaces[3]
4. **Microgrid Energy Trading (2020)**: In this paper, PPO and DDPG algorithms were compared for optimizing energy trading within microgrids. The study found that PPO, an on-policy method, provided consistent results, while DDPGoffered higher performance in specific cases. This approach highlights the balance between stability and optimization, relevant for tasks like 2048's large state space.[4]
5. **Reinforcement Learning Applied to Games (2020)**: This paper provided a comprehensive overview of reinforcement learning in gaming environments, including AlphaGo and AlphaZero. It covered foundational RL methods like Q-learning and policy gradients, discussing how RL systems can optimize strategy development. These methods are directly applicable to the 2048 problem, especially in managing exploration and exploitation in dynamic states.[5]

5. Model architecture/Mathematical Formulation of the proposed Approach(Equations and flow chart)

In this comparison, we analyzed three distinct Reinforcement Learning (RL) algorithms: Q-Learning, Deep Q-Network (DQN), and Monte Carlo Tree Search (MCTS). Here, we will outline the mathematical formulations and architecture for each algorithm.

I. Q-Learning: Mathematical Formulation

Q-Learning is a model-free RL algorithm that seeks to learn the optimal policy by estimating the Q-values for each state-action pair.

Mathematical Formulation:

- **Q-Value Update Rule**: $Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$
  - s: Current state
  - a: Current action
  - r: Reward received after taking action a in state s
  - s': Next state after taking action a in state s
  - α: Learning rate
  - γ: Discount factor

Flow Chart:

1. **Initialize**: Q-table with zeros for all state-action pairs.
2. **Loop** until convergence:
   - Choose an action a based on an exploration policy (e.g., ϵ-greedy).
   - Take action a in state s, observe reward r and next state s'.
   - Update $Q(s, a)$ using the update rule above.
3. **Convergence Check**: Repeat until a predefined number of episodes or convergence criteria are met.

II. Deep Q-Network (DQN): Model Architecture

DQN replaces the Q-table with a neural network that approximates the Q-value function. This neural network receives the state s as input and outputs the Q-values for all possible actions.

Mathematical Formulation:

- **Loss Function**: $L(\theta) = \mathbb{E}_{s,a,r,s'} \left[\left(y - Q(s, a; \theta) \right)^2 \right]$
  - Where $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$ (target Q-value)
  - θ: Weights of the current network
  - θ^-: Weights of the target network (updated periodically)

Architecture:

- **Input**: State s
- **Layers**: Fully connected layers with ReLU activation functions
- **Output**: Q-values for each action

Flow Chart:

1. **Initialize**: DQN with weights θ and target network with weights θ^-.
2. **Loop** until convergence:
    - Choose action a using an ϵ-greedy policy.
    - Execute action a, observe reward r and next state s'.
    - Store transition (s, a, r, s') in replay buffer.
    - Sample mini-batches from the replay buffer.
    - Calculate the loss using the loss function.
    - Update weights θ using gradient descent.
    - Periodically update θ^- with θ.
3. **Convergence Check**: Repeat until convergence or a fixed number of episodes.

III. Monte Carlo Tree Search (MCTS): Model Formulation

MCTS is a planning-based algorithm that builds a search tree and simulates multiple game moves to estimate the best possible action.

Mathematical Formulation:

- **Selection Rule** (Upper Confidence Bound 1, UCB1): $a^* = \arg\max_a \left[Q(s, a) + c\sqrt{\frac{\ln(N(s))}{N(s,a)}} \right]$
    - $Q(s, a)$: Estimated value of taking action a in state s
    - $N(s)$: Number of times state s has been visited
    - $N(s, a)$: Number of times action a has been taken in state s
    - c: Exploration constant

Flow Chart:

1. **Selection**: Starting from the root, recursively select actions based on UCB1 until a leaf node is reached.
2. **Expansion**: If the leaf node is not terminal, add one or more child nodes to the tree.
3. **Simulation**: Play out a sequence of random actions from the newly added child node to a terminal state.
4. **Backpropagation**: Update the Q-values of the visited nodes based on the reward obtained in the simulation.
5. **Repeat**: Until a specified number of simulations have been completed.

Flow Chart Representation

I. Q-Learning:

Start → Initialize Q-Table → Loop: Choose Action → Take Action → Observe Reward & Next State → Update Q-Table → Repeat → Converged? → End

II. DQN:

Start → Initialize DQN → Loop: Choose Action → Take Action → Observe Reward & Next State → Store in Replay Buffer → Sample Mini-Batch → Calculate Loss → Update Weights → Repeat → Converged? → End

III. MCTS:

Start → Build Root Node → Loop: Select Action → Expand Tree → Simulate to Terminal State → Backpropagate → Repeat Simulations → Choose Best Action → End

Summary of Mathematical Formulations

Q-Learning: Uses a tabular update rule to iteratively refine state-action values.

DQN: Uses a neural network to approximate Q-values, with a loss function for training.

MCTS: Uses a search tree with a UCB-based selection rule to simulate and backpropagate rewards.

These mathematical formulations and flowcharts describe the inner workings of each algorithm and provide a clear understanding of how each one approaches solving the 2048 Tiles Problem.

6. Evaluation Metrics

I. Game Score (Cumulative Reward)

The total score accumulated during a single game. As defined before the game score is the sum of the values of the merged tiles across all moves made. A higher score will indicate that the agent merged more tiles effectively. In terms of DQN and Q-learning this will be how well the model learns to maximize rewards. For MCTS, this will be the measure of effectiveness in tree-based simulations for finding high-reward sequences. This metric helps identify which model has better gameplay strategies.

Which accuracy/error measurements do you plan to use?

II. Win Rate (Percentage of Wins)

The win rate is defined as the percentage of episodes in which the model achieves the 2048 tile or higher.

$$Win\ Rate = (Total\ Episodes\ Played\ /\ Number\ of\ Wins\ (2048\ tiles\ or\ higher)) \times 100$$

This metric is a measure of consistency, a higher win rate indicates that the model accurately learns and applies an optimal strategy i.e. a higher win rate indicates a more robust and generalizable strategy, especially if evaluated over a large number of episodes.

III. Episode Length (Number of Moves per Game)

The total number of moves the agent makes before the game ends either by losing the game or achieving the max tile value. This metric is an efficiency indicator implying a shorter game length may indicate that the agent reached high tiles quickly and efficiently using one of the best possible strategies while a longer game length could indicate excessive exploration or inefficient game strategies. In other words this metric considers the trade-off between exploration and efficiency in decision-making across the models.

IV. Training Time (Computation Time)

The total time taken to train the model for DQN/Q-learning is the training time or computation time of those models. While for MCTS this is the time spent per move In other words, considering the DQN model, the training time measures the time to train the neural network with a large number of episodes. Considering Q-learning this is how quickly the agent converges when updating Q-tables. And finally for MCTS this is the measure of computational expense of running simulations before each move.

The training time helps to gauge an idea on which model is more computationally efficient and consequently cost effective, especially if used in real-time environments.

7. Baseline Model for Project Proposal

For the baseline model in the 2048 Tiles Problem, we plan to use a Q-learning algorithm, which will teach the agent to associate actions (moves) with rewards through a Q-table. The game's 4x4 grid will serve as the state space, with the agent receiving positive rewards for merging tiles and achieving higher scores, while penalties will be applied for invalid moves. Using the ε-greedy method, the agent will explore different strategies early in training and shift toward exploiting the best-known strategy as learning progresses. This baseline will provide essential performance benchmarks for comparing more advanced algorithms such as Deep Q-Networks (DQN) and Monte Carlo Tree Search (MCTS). It will allow us to evaluate how well the basic Q-learning approach can handle the game's randomness and help identify areas for improvement. The ultimate goal is to track improvements in the agent's ability to reach higher tiles, maximize the score, and develop efficient game strategies, thus laying the groundwork for future algorithmic enhancements.

8. Roles and Contributions of each group member

Algorithm Implementation, Research, Visualization, and Analysis Split:

1. Raj Trikha
   Role: Q-Learning Algorithm Implementation, Research, & Visualization
   Responsibilities:
   - Implement the Q-learning algorithm, focusing on state-action pairs and Q-table management.
   - Research related work and provide detailed insights into Q-learning applications in game environments.
   - Compare Q-learning with DQN and MCTS in terms of average score, win rate, and training time.
   - Create visualizations (charts/graphs) for Q-learning performance metrics and contribute to final project documentation.

2. Shweta Shekhar
   Role: DQN Algorithm Implementation, Research, & Visualization
   Responsibilities:
   - Implement the DQN algorithm, integrating experience replay and target networks.
   - Research deep reinforcement learning approaches and their effectiveness in dynamic environments.
   - Conduct a comparative analysis of DQN against Q-learning and MCTS, focusing on performance metrics like score improvement and efficiency.
   - Create visualizations to depict DQN's learning curve and convergence rate, contributing to analysis and documentation.

3. Rachit Pathak
   Role: MCTS Algorithm Implementation, Research, & Analysis
   Responsibilities:
   - Implement the MCTS algorithm, focusing on simulations and reward backpropagation.
   - Research applications of MCTS in decision-making and game-solving problems.
   - Lead the comparison of MCTS with Q-learning and DQN, particularly focusing on decision accuracy, computational cost, and win rate.
   - Contribute to data analysis and assist in creating visual representations of MCTS performance (tree simulations, win rates).

9. References

[1].Kaundinya, V., Jain, S., Saligram, S., Vanamala, C. K., & Avinash, B. (2018). Game Playing Agent for 2048 using Deep Reinforcement Learning. *Proceedings of the 3rd National Conference on Image Processing, Computing, Communication, Networking and Data Analytics (NCICCNDA 2018)*. AIJR Publisher. DOI: https://doi.org/10.21467/proceedings.1.57

[2].Atlantis Press. (n.d.). Comparison of Three Deep Reinforcement Learning Algorithms for Continuous Control Tasks.

Retrieved from https://www.atlantis-press.com/proceedings/dai-23/125998095

[3].Zhang, Y., & Liu, H. (2019). A Comparative Performance Study of Reinforcement Learning Algorithms for a Continuous Space Problem. *ResearchGate*.

Retrieved from https://www.researchgate.net/publication/336588359_A_Comparative_Performance_Study_of_Reinforcement_Learning_Algorithms_for_a_Continuous_Space_Problem

[4].IEEE Xplore. (2021). Comparison of Deep Reinforcement Learning Algorithms in Enhancing Energy Trading in Microgrids.

Retrieved from https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9429565

[5].SpringerLink. (2020). Reinforcement Learning Applied to Games (2020). *SN Applied Sciences*, 2(2560). DOI: https://doi.org/10.1007/s42452-020-2560-3